

Gazebo 플러그인

IMU · LiDAR · Camera 센서 시뮬레이션

cchyun@gmail.com

Part 1. Gazebo 시뮬레이터 기초

- 실물 로봇으로만 테스트하는 것은 비효율적이고 위험 — 시뮬레이터가 대안
 - 반복 가능성 및 효율성
 - 동일한 환경에서 알고리즘을 수천 번 반복 테스트 가능
 - 안전성
 - 복잡한 내비게이션·고속 주행 알고리즘 테스트 중 벽 충돌 → 물리 파손 없음
 - 비용 절감
 - 값비싼 실물 센서·하드웨어 없이 가상 로봇 기동 가능

- 시뮬레이터는 현실을 완벽히 재현하지 못함 — 이 간극을 이해하고 대비해야 함
 - 물리적 불확실성
 - 현실 센서에는 노이즈가 있고, 환경은 시물보다 훨씬 역동적
 - 오차 누적 (드리프트)
 - 시뮬레이터의 오도메트리(Odometry)는 시간이 지나며 실제 위치와 차이 발생
 - 바퀴 미끄러짐 (Wheel Slip)
 - 물리 엔진으로 모사하지만 실제 노면 상태를 완벽히 재현하기 어려움

- 현재 Gazebo는 두 계열로 나뉨 — ROS 2 버전에 맞는 선택이 중요

- Gazebo Classic (v11 등)

- 오랫동안 사용되어 온 안정적인 버전
- 튜토리얼·기존 패키지 호환성 우수

- Gazebo Ignition (신규 명칭: Gazebo)

- 완전히 새로 설계된 차세대 시뮬레이터
- 릴리스: **Fortress, Harmonic, Ionic**
- 최신 기능·렌더링 성능 우수

- Gazebo는 World → Model → Link → Joint 의 계층 구조로 구성
 - World
 - 로봇이 움직이는 전체 가상 공간(스테이지)
 - 조명, 지면(Ground Plane), 장애물 포함
 - Model
 - 로봇 본체나 상자 같은 개별 객체
 - 여러 개의 Link와 Joint로 구성
 - Link
 - 로봇의 뼈대·부품 — 질량, 충돌 영역(Collision), 시각 형태(Visual) 보유
 - Joint
 - 두 Link를 연결하는 관절 — 회전(Continuous/Revolute), 슬라이딩(Prismatic) 등

- 시뮬레이션 정밀도는 Physics 엔진 파라미터로 제어
 - ODE / Bullet — 물리 현상 계산 엔진
 - `max_step_size` — 물리 계산 최소 시간 단위 (보통 0.001초), 시뮬레이션 정밀도 결정
 - `real_time_factor` — 실제 시간 대비 시뮬레이션 속도 비율 (1.0 = 현실과 동일)
- 환경 파일 구조 (.world / .sdf)
 - SDF (Simulation Description Format)
 - Gazebo에서 World와 Model을 설명하는 XML 기반 표준 형식
 - `<sdf>` → `<world>` → 물리 엔진 설정, 조명, 모델 배치 순서로 정의

Part 2. Gazebo 플러그인과 제어기

- 플러그인은 가상 세계의 물리 데이터를 ROS 2 메시지로 변환하는 "브릿지(Bridge)"
 - 동작 원리
 - 시뮬레이터 내에서 가상의 레이저(Ray)가 물체에 닿는 거리를 물리 엔진이 계산
 - Ray/LiDAR 플러그인이 이를 가로채어 ROS 2의 `/scan` 토픽으로 발행
 - 센서 시뮬레이션
 - IMU, Camera, LiDAR 등 실제 하드웨어 센서가 하는 일을 소프트웨어 코드가 대신 수행
 - 로봇의 '눈'과 '귀'를 가상으로 구현

- 로봇 바퀴를 실제로 구동하려면 제어기(Controller)가 필요
 - ros2_control
 - ROS 2에서 로봇 하드웨어를 제어하기 위한 **실시간 프레임워크**
 - diff_drive_controller — 차동 구동(Differential Drive) 로봇용 제어 패키지
 - 입력: `/cmd_vel` (속도 명령) 구독
 - 출력 1: 각 바퀴에 필요한 토크·속도를 계산하여 물리 엔진에 전달
 - 출력 2: 바퀴 회전수로 현재 위치인 **오도메트리**(`/odom`) 계산 후 발행

Part 3. 센서 플러그인 설정

- LiDAR 플러그인은 레이저 스캔 데이터를 생성하여 주변 장애물과의 거리를 측정
 - URDF/SDF 추가 방법
 - `<sensor>` 태그 내부에 `type="ray"` 또는 `type="gpu_lidar"` 지정
 - `<plugin>` 태그의 `filename` 을 `libgazebo_ros_ray_sensor.so` 로 설정
 - 핵심 파라미터
 - `frameName` — 메시지 헤더 좌표계 이름 (URDF 링크 이름과 일치해야 함)
 - `topicName` — 데이터 발행 ROS 토픽 이름 (예: `/scan`)
 - `min/maxAngle` — 수평 측정 범위(라디안) (예: $-3.14 \sim 3.14 = 360$ 도)
 - `rangeResolution` — 거리 측정 최소 분해능
 - `samples` — 한 주기 스캔 시 생성할 레이저 빔 개수
 - 발행 토픽: `/scan` → `sensor_msgs/LaserScan` 타입

- IMU 센서는 로봇의 관성 정보(가속도, 각속도, 방향)를 제공
 - URDF/SDF 추가 방법
 - 센서를 부착할 링크(예: `imu_link`) 아래에 `<sensor type="imu">` 정의
 - `libgazebo_ros_imu_sensor.so` 플러그인 로드
 - 핵심 파라미터
 - `frameName` — IMU 데이터 기준 좌표계 이름
 - `topicName` — IMU 데이터 발행 토픽 이름 (예: `/imu`)
 - `updateRate` — 데이터 업데이트 주기 (Hz)
 - `gaussianNoise` — 센서 값에 추가할 가우스 노이즈 표준 편차
 - 노이즈 추가 이유
 - 실제 물리 센서의 불완전성·오차를 모사 → 현실과 유사한 데이터
 - EKF(Extended Kalman Filter) 등 필터 알고리즘 실질적 테스트 가능

- 카메라 플러그인은 로봇 시점의 영상 데이터를 렌더링하여 제공

- URDF/SDF 추가 방법

- `<sensor type="camera">` 태그 사용
- `libgazebo_ros_camera.so` 플러그인 추가

- 핵심 파라미터

- `frameName` — 카메라 렌즈의 위치·방향 좌표계 이름
- `topicName` — 이미지 데이터 발행 토픽 (예: `/image_raw`)
- `width / height` — 이미지 가로·세로 해상도 (픽셀)
- `focalLength` — 렌즈 초점 거리
- `distortion` — 렌즈 왜곡 계수 (실제 카메라 렌즈 특성 구현)

- 깊이 카메라(Depth Camera)

- RGB 영상 + 거리 정보 포함 깊이 맵 동시 제공
- `sensor_msgs/Image` (깊이 이미지) + `sensor_msgs/PointCloud2` (3D 점군) 발행

Part 4. 실습: URDF에 Gazebo 플러그인 추가

1. 워크스페이스 루트로 이동

```
cd ~/Workspace/ros2-ws
```

2. Gazebo 및 ROS2 연동 패키지 설치

```
sudo apt install ros-humble-gazebo-ros-pkgs ros-humble-teleop-twist-keyboard
```

3. 빌드 (처음 또는 코드 수정 후)

```
colcon build
```

```
source install/setup.bash
```

Step 1-1. Gazebo 물리 속성 및 색상 추가

- `urdf.md`의 `turtlebot.xacro`를 기반으로 Gazebo 물리 시뮬레이션 및 센서 동작 플러그인 추가
- Gazebo 전용 속성(마찰계수, 색상)을 별도 xacro 파일로 분리 관리

Step 1-1. Gazebo 물리 속성 및 색상 추가

- `my_robot_description/urdf/turtlebot_gazebo.xacro` :

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- ===== Physical Properties ===== -->
  <gazebo reference="base_link">
    <mu1>0.5</mu1>
    <mu2>0.5</mu2>
    <material>Gazebo/LighgGray</material>
  </gazebo>

  <gazebo reference="wheel_left_link">
    <mu1>1.0</mu1>
    <mu2>1.0</mu2>
    <material>Gazebo/Black</material>
  </gazebo>

  <gazebo reference="wheel_right_link">
    <mu1>1.0</mu1>
    <mu2>1.0</mu2>
    <material>Gazebo/Black</material>
  </gazebo>

  <gazebo reference="caster_back_link">
    <mu1>0.1</mu1>
    <mu2>0.1</mu2>
    <material>Gazebo/LighgGray</material>
  </gazebo>

  <gazebo reference="base_scan">
    <material>Gazebo/DarkGrey</material>
  </gazebo>

  <gazebo reference="camera_link">
    <material>Gazebo/Blue</material>
  </gazebo>

</robot>
```

Step 1-1. xacro include 및 핵심 개념

- `turtlebot.xacro`의 `<robot>` 태그 내부, 기존 `camera_link` 아래에 include 추가

```
<!-- Gazebo Property -->  
<xacro:include filename="turtlebot_gazebo.xacro" />
```

- **핵심 개념:**

- `<gazebo reference="링크명">` 은 해당 링크의 Gazebo 전용 속성 설정.
- `mu1` / `mu2`: 마찰계수 — `mu1` 은 주요 마찰 방향, `mu2` 는 수직 방향 (보통 동일하게 설정).
- `material`: Gazebo 시각화 색상.
- xacro의 `<xacro:include>` 를 사용하여 Gazebo 관련 설정을 별도 파일로 모듈화 가능.

Step 1-1-1. 중간 확인 — 빌드

- 1-1까지 진행 후 Gazebo 정상 스폰 여부 확인
- 1) 빌드

```
cd ~/Workspace/ros2-ws  
colcon build --packages-select my_robot_description  
source install/setup.bash
```

- 2) Gazebo 실행 및 로봇 스폰

터미널 1: Gazebo 실행

```
gazebo --verbose \  
-s libgazebo_ros_init.so \  
-s libgazebo_ros_factory.so
```

터미널 2: robot_state_publisher 실행

```
ros2 run xacro xacro src/my_robot_description/urdf/turtlebot.xacro > src/my_robot_description/urdf/turtlebot_from_xacro.urdf
```

```
ros2 run robot_state_publisher robot_state_publisher ./src/my_robot_description/urdf/turtlebot_from_xacro.urdf
```

터미널 3: Gazebo에 로봇 스폰

```
ros2 run gazebo_ros spawn_entity.py \  
-topic robot_description \  
-entity turtlebot \  
-z 0.1
```

3) 확인 항목

확인 사항	기대 결과
본체 색상	밝은 회색 (<code>Gazebo/LighgGray</code>)
바퀴 색상	검정색 (<code>Gazebo/Black</code>)
캐스터 색상	밝은 회색 (<code>Gazebo/LighgGray</code>)
LiDAR 색상	어두운 회색 (<code>Gazebo/DarkGrey</code>)
카메라 색상	파란색 (<code>Gazebo/Blue</code>)
바닥 안착	로봇이 바닥에 떨어져 정지

Step 1-2. 차동 구동 플러그인 추가

- `turtlebot_gazebo.xacro`에 Diff Drive 플러그인 추가

```
<!-- ===== DIFF DRIVE PLUGIN ===== -->
<gazebo>
<plugin name="diff_drive" filename="libgazebo_ros_diff_drive.so">
  <update_rate>50</update_rate>

  <!-- Left/Right Wheel Joints -->
  <left_joint>wheel_left_joint</left_joint>
  <right_joint>wheel_right_joint</right_joint>

  <!-- Wheel Separation & Diameter (meters) -->
  <wheel_separation>0.160</wheel_separation>
  <wheel_diameter>0.066</wheel_diameter>

  <!-- Limits -->
  <max_wheel_acceleration>1.0</max_wheel_acceleration>

  <!-- Topic Configuration -->
  <topic>cmd_vel</topic>
  <odometry_topic>odom</odometry_topic>
  <publish_odom>true</publish_odom>
  <publish_odom_tf>true</publish_odom_tf>

  <!-- Coordinate Frames -->
  <frame_name>odom</frame_name>
  <child_frame_name>base_footprint</child_frame_name>

  <!-- Torque Limit -->
  <max_wheel_torque>10</max_wheel_torque>
</plugin>
</gazebo>
```

Step 1-2. 빌드 및 실행, 토픽 확인

- 1) 빌드 및 실행

```
cd ~/Workspace/ros2-ws
colcon build --packages-select my_robot_description
source install/setup.bash
```

Gazebo 실행 및 로봇 스폰: 1-1-1 단계와 동일

- 2) 토픽 확인

```
ros2 topic list
```

토픽	설명
/cmd_vel	속도 명령 (입력)
/odom	주행 궤적 (출력)

• 3) 키보드 조작

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

키	동작
i	전진
,	후진
j	좌회전 (제자리)
l	우회전 (제자리)
k	정지
q/z	최고 속도 증가/감소

Step 1-2. Gazebo 창에서 로봇 이동 확인

- **핵심 개념:** `libgazebo_ros_diff_drive.so` 는 `/cmd_vel` 구독 후 바퀴에 토크를 가하고 `/odom` 발행.
 - `wheel_separation`: 좌우 바퀴 중심 간 거리($0.080 * 2 = 0.160$).
 - `wheel_diameter`: 바퀴 지름($0.033 * 2 = 0.066$).
 - `max_wheel_acceleration`: 바퀴 최대 가속도 제한.

Step 1-3. LiDAR 플러그인 추가

- `turtlebot_gazebo.xacro`에 LiDAR 센서 플러그인 추가

```
<!-- ===== LASER PLUGIN ===== -->
<gazebo reference="base_scan">
  <sensor name="laser" type="ray">
    <always_on>true</always_on>
    <visualize>true</visualize>
    <update_rate>10</update_rate>
    <ray>
      <scan>
        <horizontal>
          <samples>360</samples>
          <resolution>1</resolution>
          <min_angle>-3.14</min_angle>
          <max_angle>3.14</max_angle>
        </horizontal>
      </scan>
      <range>
        <min>0.12</min>
        <max>3.5</max>
        <resolution>0.015</resolution>
      </range>
    </ray>
    <plugin name="laser_controller" filename="libgazebo_ros_ray_sensor.so">
      <ros>
        <remapping>~/out:=scan</remapping>
      </ros>
      <output_type>sensor_msgs/LaserScan</output_type>
      <frame_name>base_scan</frame_name>
    </plugin>
  </sensor>
</gazebo>
```

Step 1-3. 빌드 및 토픽 확인

- 1) 빌드 및 실행

```
cd ~/Workspace/ros2-ws
colcon build --packages-select my_robot_description
source install/setup.bash
```

- Gazebo 실행 및 로봇 스폰: 1-2 단계와 동일

- 2) 토픽 확인

```
ros2 topic list
```

토픽	설명
/scan	LiDAR 거리 데이터 (출력)

```
# 데이터 한 번 출력
ros2 topic echo /scan --once
```

Step 1-3. RViz2로 시각화

- 3) RViz2로 시각화 (선택)

```
rviz2
```

항목	설정값
Add → LaserScan	Topic: <code>/scan</code>

- 핵심 개념: `libgazebo_ros_ray_sensor.so` 는 Gazebo의 ray 센서를 `sensor_msgs/LaserScan` 으로 변환하여 `/scan` 토픽에 발행.
 - `samples=360` 시 1도 간격으로 360개 거리값 측정.

Step 1-4. IMU 플러그인 추가

- `turtlebot_gazebo.xacro` 에 IMU 센서 플러그인 추가

```
<!-- ===== IMU PLUGIN ===== -->
<gazebo reference="imu_link">
  <sensor name="imu_sensor" type="imu">
    <always_on>true</always_on>
    <update_rate>200</update_rate>
    <imu>
      <angular_velocity>
        <x>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>2e-4</stddev>
          </noise>
        </x>
        <y>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>2e-4</stddev>
          </noise>
        </y>
        <z>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>2e-4</stddev>
          </noise>
        </z>
      </angular_velocity>
      <linear_acceleration>
        <x>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>1.7e-2</stddev>
          </noise>
        </x>
        <y>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>1.7e-2</stddev>
          </noise>
        </y>
        <z>
          <noise type="gaussian">
            <mean>0.0</mean>
            <stddev>1.7e-2</stddev>
          </noise>
        </z>
      </linear_acceleration>
    </imu>
    <plugin name="imu_plugin" filename="libgazebo_ros_imu_sensor.so">
      <ros>
        <remapping>~/out:=imu</remapping>
      </ros>
      <frame_name>imu_link</frame_name>
    </plugin>
  </sensor>
</gazebo>
```

Step 1-4. 빌드 및 토픽 확인

- 1) 빌드 및 실행

```
cd ~/Workspace/ros2-ws
colcon build --packages-select my_robot_description
source install/setup.bash
```

- Gazebo 실행 및 로봇 스폰: 이전 단계와 동일

- 2) 토픽 확인

```
ros2 topic list
```

토픽	설명
/imu	IMU 관성 데이터 (출력)

```
# 데이터 한 번 출력
ros2 topic echo /imu --once
```

- 핵심 개념: `libgazebo_ros_imu_sensor.so` 는 Gazebo IMU 센서를 `sensor_msgs/Imu` 로 변환하여 `/imu` 토픽에 발행.

Step 1-5. Camera 플러그인 추가

turtlebot_gazebo.xacro에 Camera 센서 플러그인 추가

```
<!-- ===== CAMERA PLUGIN ===== -->
<gazebo reference="camera_link">
  <sensor name="camera" type="camera">
    <always_on>true</always_on>
    <visualize>true</visualize>
    <update_rate>30</update_rate>
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image>
        <width>640</width>
        <height>480</height>
        <format>R8G8B8</format>
      </image>
      <clip>
        <near>0.05</near>
        <far>8.0</far>
      </clip>
    </camera>
    <plugin name="camera_controller" filename="libgazebo_ros_camera.so">
      <ros>
        <remapping>/camera/image_raw:=/image_raw</remapping>
        <remapping>/camera/camera_info:=/camera_info</remapping>
      </ros>
      <camera_name></camera_name>
      <frame_name>camera_link</frame_name>
    </plugin>
  </sensor>
</gazebo>
```

Step 1-5. 빌드 및 토픽 확인

- 1) 빌드 및 실행

```
cd ~/Workspace/ros2-ws
colcon build --packages-select my_robot_description
source install/setup.bash
```

- Gazebo 실행 및 로봇 스폰: 이전 단계와 동일

- 2) 토픽 확인

```
ros2 topic list
```

토픽	설명
/image_raw	카메라 영상 (출력)
/camera_info	카메라 내부 파라미터 (출력)

- 핵심 개념: libgazebo_ros_camera.so 는 Gazebo 카메라 센서를 sensor_msgs/Image 로 변환.

Step 1-7. 최종 빌드



```
cd ~/Workspace/ros2-ws  
colcon build --packages-select my_robot_description  
source install/setup.bash
```